

Computational Complexity Analyses of Modular Arithmetic for RSA Cryptosystem

Chia-Long Wu^{*}, Der-Chyuan Lou, and Te-Jen Chang

^{*}Department of Aviation & Communication Electronics, Chinese Air Force Institute of Technology, Kaohsiung 82042, Taiwan.

E-mail: chialongwu@seed.net.tw

Department of Electrical Engineering, Chung Cheng Institute of Technology, National Defense University, Tahsi, Taoyuan 33509, Taiwan.

E-mail: dclou@ccit.edu.tw

Abstract

Modular arithmetic is fundamental to several public-key cryptography systems such as the RSA encryption system, as well as the most dominant part of the computation performed. The operation is time-consuming for large operands. Many relevant papers about computer security are issued in many reports and journals to describe how to reduce the computational complexities in the cryptosystems. In this paper, we describe the modular arithmetics and some improved algorithms. These algorithms by using binary method, common multiplicand multiplication method (CMM method), signed-digit recoding method, etc. will be depicted respectively. We will analyze and compare the complexity of various algorithms for computing large modular exponentiation.

Keywords : Complexity analysis , modular arithmetic, computer security, public-key cryptosystem, exponentiation.

1. Introduction

Data encryption is more and more needed in the world-wide computer network environment. RSA cipher is the most common cryptography currently. Since the pioneering work of public-key cryptosystem has been performed by [6], a lot of investigations and system developments on this subject have been done in the past decade, such as RSA algorithm [5], elliptic curve cryptography [7], etc. RSA (Rivest, Shamir, and Adleman) is one of the most widely used public-key cryptosystems and modular exponentiation is the main operation to RSA-based public-key cryptosystems. It is performed by successive modular multiplications. For fast cryptosystems, one needs to reduce total number of modular multiplications required [9].

In the rest of this paper, we will present and compare methods that attempt to minimize the numbers of multiplications. We will review the modular arithmetics including modular multiplication, modular squaring, modular exponentiation, and modular multi-exponentiation in Section 2. In Section 3, we present some modular exponent arithmetics, such as binary method, CMM method, exponent-folding exponentiation, signed-digit

^{*} This is the corresponding author of this paper.

recoding method, and high radix method. In Section 4, we will make tables and some experiments for the comparisons of computational complexities. Finally, some concise conclusions are given in Section 5.

2. Modular Arithmetic

RSA cryptography (named for its inventors Rivest, Shamir, and Adelman) exploits properties and interrelations of numbers, constructed as large powers of huge numbers. Through a neat mathematical trick called modular arithmetic, the computer avoids working with the huge numbers themselves. As we know modular exponentiation is important in the modular arithmetic, it can be considered as a series of modular square-and-multiply operation. The squaring operations can be regarded as a case of multiplication operations. So we describe modular multiplication, modular squaring, and modular exponentiation respectively in Section 2.1 – Section 2.4.

2.1 Modular Multiplication

The modular multiplication multiplies the input operands C and D and reduces the product modulo N . That is to say, it computes $Q \equiv CD \bmod N$. In 1985, Montgomery proposed an efficient algorithm for modular multiplications without trial division [13]. This algorithm does not perform division and multiplication, but uses digit operations of shift and addition. The Montgomery multiplication method is described as follows.

Algorithm 1 (Montgomery's algorithm)

Input: C, D, N

/ $CD \bmod N = tR \bmod N$. */*

*/*That means $t = CDR^{-1} \bmod N$. */*

Output: t

/ C and D are the factors of $R \bmod N$. */*

begin

$u[0] = 0;$

/ R is 2^n and N is an odd n -bit integer. */*

for ($i = 0, 1, \dots, n - 1$)

/ t is a factor of R^{-1} . */*

parbegin

$q_i = (u[i] + a_i D) \bmod 2;$

/ q_i is a result of modulo 2. */*

$u[i + 1] = \frac{(u[i] + a_i D + q_i N)}{2};$

parend;

if $u[n] < N$ **then** $t = u[n]$

else $t = u[n] - N;$

end.

This algorithm computes the modular multiplication, $CD \bmod N \equiv tR \bmod N$ (i.e. $t \equiv CDR^{-1} \bmod N$), while avoiding division by N . These three inputs A, B and N are n -bit integers. The output is t with a factor of R^{-1} , where R is 2^n . The q_i is a result of modulo 2 as can be verified by simply checking the last bit of $u[i] + a_i D$. The shift-to-right operation can be considered as division by 2. More descriptions of Montgomery's algorithm are depicted in [13, 14].

2.2 Modular Squaring

Computations involving multiplying and squaring large integers are key factors in the performance of many public key cryptosystems. In [2], three algorithms (squaring computation which uses the standard multiplication algorithm, the Guajardo-Parr squaring algorithm, and the Yang-Hseih-Laih's algorithm) are compared by the numbers of the modular squaring. More descriptions will be depicted in [2]. The performance of the Yang-Hseih-Laih's algorithm is nearly 2.52 and 1.50 times faster than squaring computation by using the

standard multiplication algorithm and the Guajardo-Parr squaring algorithm. The Yang-Hseih-Laih's algorithm, which is the fastest among three algorithms, is described as follows.

Algorithm 2 (Yang-Hseih-Laih's algorithm)

Input: $A = (a_{n-1}, a_{n-2}, \dots, a_1, a_0)_b$

Output: $D = (d_{2n-1}, d_{2n-2}, \dots, d_1, d_0)_b = A^2$

begin

$(d_{2n-1}, d_{2n-2}, \dots, d_1, d_0)_b = (0, 0, \dots, 0, 0)_b;$

for $i = 0$ **to** $n-1$

$c = 0$

for $j = i + 1$ **to** $n-1$

$(c, s) \leftarrow d_{i+j} + a_j * a_i + c;$

$d_{i+j} = s;$

$d_{i+n} = c;$

$D = 2D$ (shift D left 1 bit);

$c = 0;$

for $i = 0$ **to** $n-1$

$(c, s) = D_{2i} + a_i * a_i + c, d_{2i} \leftarrow s;$

$(c, s) \leftarrow D_{2i+1} + c, d_{2i+1} \leftarrow s;$

end.

2.3 Modular Exponentiation

The modular exponentiation algorithm performs in conjunction with the modular multiplication algorithm as presented in Section 2.1. The algorithm comes with the advantage of the Montgomery multiplication algorithm without trial division. For exponentiation computation, the exponent is scanned from left to right bit by bit [13]. The Lai-Chang's algorithm is described as follows.

Algorithm 3 (Lai-Chang's algorithm)

Input: A, X, N

$/* A$ is the factor of $R \bmod N. */$

Output: $D = MA(C, 1)$

$/* R$ is $2^n. */$

begin

$C = R - N; Q = AR \bmod N;$

$/* N$ is an odd n -bit integer $*/$

$k = h - 1;$

$/* X$ is in h bits $*/$

if $(k \geq 0)$

if $x_k = 1$, then $C = MA(A, C);$

if $k \geq 1$, then $C = MA(C, C);$

$k = k - 1.$

end.

2.4 Modular Multi-Exponentiation

In the computation of multi-exponentiation, there are two or more bases and exponents. Unlike the computation of exponentiation, the multiplication operations need to be performed if any scanned digit of exponent presentation is non-zero. Thus it is important to consider the corresponding bits of other exponents. The multiplication operation cannot be removed unless all corresponding bits of other exponents are also zero.

The computation time of a multi-exponentiation computation depends on the numbers of square-and-multiply operations [13]. In other words, those exponents with fewer non-zero bits might introduce a better performance. The Yen-Laih's modular multi-exponentiation is given as follows. In Algorithm 4, the exponent with fewer bits, X or Y , will be padded with bits of zero from the most significant bit to make X and Y the same length, h bits.

Algorithm 4 (Yen-Laih's algorithm)

Input: A, B, X, Y, N

$/* X$ and Y have the same length " h bits". $*/$

Output: $C \equiv A^X B^Y \bmod N$

begin

$X = (x_{h-1}, x_{h-2}, \dots, x_0); Y = (y_{h-1}, y_{h-2}, \dots, y_0);$

```

/* h = Max(log2X, log2Y*/
C = 1; Q ≡ A*B mod N;
k = h - 1;
case (xk, yk)
  (1, 0): C ≡ C*A mod N,
  (0, 1): C ≡ C*B mod N,
  (1, 1): C ≡ C*Q mod N,
endcase;
if k >= 1, then C ≡ C*C mod N;
      k = k - 1;
if (k >= 0), then do case (xk, yk),
      otherwise, Output C.
end.

```

3. Modular Exponentiation Arithmetic

As we know, the modular exponentiation, $M^e \bmod N$, where M is a plaintext, e is an exponent, and N is a modulus, is one of the most important operations in most public key cryptosystems [15, 16]. It is more complicated and time-consuming. So many researchers are devoted to finding ways to reduce the time such as binary method, common-multiplicand multiplication method, signed-digit recoding method, and high-radix method, etc. We introduce these methods respectively as follows.

3.1 Binary Method

RSA (Rivest, Shamir, and Adleman) method [5] is one of the most widely used public-key cryptosystems. This method is performed using successive modular multiplications and time consuming for large operands [8] and the exponentiation operation is broken into a series of squaring and multiplication operations by the use of the binary method [17]. The numbers of modular multiplications strongly depend on the Hamming weight of B , where B is the exponent [1]. So many

scholars devoted to reducing the Hamming weight [17, 19] and the binary method is one of the fast and efficient methods which can improve the modular multiplications [20-23]. The binary method is given as follows [18].

Algorithm 5 (Martinez-Luna-Henriquez's algorithm)

```

Input:  $M, E, N$  /*  $M$  is a plaintext. */
Output:  $P \equiv M^E \bmod N$  /*  $E$  is an exponent. */
 $C := M; P := 1;$  /*  $N$  is a modulus. */
begin
  for  $i := 0$  to  $n-1$  do
    if  $E_i := 1$  then  $P \equiv P * C \bmod N$ 
       $C \equiv C * C \bmod N$ 
    end for;
end.

```

A simple example is given in the following to illustrate Algorithm 5.

Example 1:

$E = (10101010)_2 = (170)_{10}$ can be evaluated as the following steps.

E	Step 3 (R)	Step 4 (C)
0	1	$(M)^2 = M^2$
1	$1 * M^2 = M^2$	$(M^2)^2 = M^4$
0	M^2	$(M^4)^2 = M^8$
1	$M^2 * M^8 = M^{10}$	$(M^8)^2 = M^{16}$
0	M^{10}	$(M^{16})^2 = M^{32}$
1	$M^{10} * M^{32} = M^{42}$	$(M^{32})^2 = M^{64}$
0	M^{42}	$(M^{64})^2 = M^{128}$
1	$M^{42} * M^{128} = M^{170}$	$(M^{128})^2 = M^{256}$

3.2 Common-Multiplicand Multiplication Method

In 1993, Yen and Laih developed the technique of common-multiplicand- multiplication [22] and in 1996, Lou and Chang developed the exponent folding exponentiation method based on [24] to enhance the

performance of the famous right to left binary exponentiation algorithm. Some public cryptosystems (the Diffie-Hellman method [6], the RSA method [5], and DSA systems [25]) are also based on the CMM method [22]. CMM method is described as follows [26].

Algorithm 6 (Wu-Chang's algorithm)

Input: Exponent $E = (e_{k-1}e_{k-2}\dots e_1e_0)_2$;
/* M is message. */

Output: Ciphertext $C = M^E$;

$C_1 = C_2 = C_3 = 1$; /* $E_y = (e_y^{\frac{k}{2^n}} e_y^{\frac{k}{2^{n-1}}} \dots e_y^1)_2$ */

$S = x$;
/* k is the bit length of E and E is the exponent. */

begin

for $b = 1$ **to** $\frac{k}{2^n}$ **do**

/*scan from LSB to MSB*/

parbegin

if $(e_{excl_j}^b = 1)$ **then** $C_1 = S * C_1$;
/*multiply*/

if $(e_{excl_j+1}^b = 1)$ **then** $C_2 = S * C_2$;
/*multiply*/

if $(e_{com_j}^b = 1)$ **then** $C_3 = S * C_3$;
/*multiply*/

$S = S * S$;

parend ;

$C_1 = C_1 * C_3$;

$C_2 = C_2 * C_3$;

end.

The results of x^{E_j} and $x^{E_{j+1}}$ are kept in C_1 and C_2 respectively. More detailed descriptions of Wu-Chang's algorithm are depicted in [26].

3.3 Exponent-Folding Method

In 1996, Lou and Chang [24] proposed a fast exponentiation method by using exponent- folding exponentiation method. The basic idea of this method is to extract the common substring of the exponent E , and then save the numbers of multiplications required for the computation of common strings. Exponent-folding method is described as follows.

Algorithm 7 (Lou-Chang's algorithm)

Input: M, E_i, E_{i+1} , where $i = 1, 2, \dots, 2^n$
/* M is message. */

Output: $M^{E_i}, M^{E_{i+1}}$
/* n is the folding-time of E . */

$C_1 = C_2 = C_3 = 1$; $S = M$;
/* E_i , and E_{i+1} , each of $\frac{k}{2}$ bits */

begin

for $b = 1$ **to** $\frac{k}{2^n}$ **do**

/*scan for the exponent-folding parts*/

parbegin

if $(e_{excl_i}^b = 1)$ **then** $C_1 = S * C_1$ /*multiply*/

if $(e_{excl_i+1}^b = 1)$ **then** $C_2 = S * C_2$ /*multiply*/

if $(e_{common_i}^b = 1)$ **then** $C_3 = S * C_3$ /*multiply*/

$S = S * S$; /* square */

parend ;

$C_1 = C_1 * C_3$;

$C_2 = C_2 * C_3$;

end.

More detailed descriptions of Lou-Chang's algorithm are depicted in [24].

3.4 Signed-Digit Recoding Method

In 1993, Arno and Wheeler presented signed-digit recoding method [27]. The Arno-Wheeler's algorithm for signed-digit recoding method is depicted as

follows.

Algorithm 8 (Arno-Wheeler's algorithm)

Input: $a \in S_r$ with $\pi(a) = n$;

/ a is a redundant representation of n . */*

Output: $A(a) \in S_r$;

/ $A(a)$ denotes the action of this algorithm on a . */*

$t = 0$;

/ S_r denotes the set of all sequences:*

$a = (\dots, a_2, a_1, a_0)$ **/*

while $(\dots, a_{t+2}, a_{t+1}, a_t) \neq (\dots, 0, 0, 0)$ **do**

/ $a_i \in \{0, \pm 1, \dots, \pm(r-1)\}$ **/**

parbegin

$b = (\dots, \text{sgn}(a_t), -\text{sgn}(a_t) * r, 0, \dots, 0)$

/ nonzero at t and $t+1$ **/**

$c = a + b$;

if $c_{t+1} = 0$ **then** $a = c$;

parend;

$t = t + 1$;

end.

More detailed descriptions of Arno-Wheeler's algorithm are depicted in [22, 27].

3.5 High Radix Method

The binary method can be generalized to the high radix method which scans the digits of E expressed in radix m [10, 29]. We focus our attention to the case when $m = 2^d$. The exponent E is partitioned into k sections of d bits for $kd = n$. If d does not divide n , the exponent is padded with at most $d-1$ zeros. Let

$$F^{(i)} = [E_{id+d-1} E_{id+d-2} \dots E_{id}] = \sum_{r=0}^{d-1} E_{id+r} 2^r.$$

First, the values of $X_j = M^j \bmod N$ for $j = 2, 3, \dots, 2^d - 1$ are computed. Then, the bits of E are scanned d bits at a time from the most significant to the least significant.

At each step, the partial result is raised to the 2^d

power and multiplied with $X_{F^{(i)}}$, where $F^{(i)}$ is the value of the current bit section. High radix method is described as follows.

Algorithm 9 (Koc's algorithm)

Input: M, N, E, n, d where $n = \lfloor \log_2 E \rfloor + 1$ and $n =$

kd for $k \geq 1$.

Output: $C \equiv M^E \bmod N$.

begin

Set $X_0 = 1$ and $X_1 = M$.

Repeat

for $j = 2, 3, \dots, 2^d - 1$.

$X_j \equiv (X_{j-1} * M) \bmod N$.

Repeat

for $i = k - 2, k - 3, \dots, 0$.

$$F^{(i)} = \sum_{r=0}^{d-1} E_{id+r} 2^r.$$

Set $C = X_{F^{(k-1)}}$.

Repeat

for $i = k - 2, k - 3, \dots, 0$.

Repeat

for $j = 0, 1, d - 1$.

$$C \equiv C * C \bmod N$$

if $F^{(i)} \neq 0$ **then** $C \equiv C * X_{F_i} \bmod N$.

Halt.

end.

More detailed descriptions of Koc's algorithm are depicted in [29].

4. Complexity Analyses

In this section, the performance of many methods described as above will be compared and several presented methods [13-15] are also compared. As we know, the squaring operations can be regarded as a case of multiplication operations and for clarity, the

modular reductions are omitted. In order to measure the speed of the modular multiplications, modular squarings, and modular exponentiations, etc., we use the numbers of modular multiplications to express the speed-up efficiency [15]. As mentioned in the previous section, it needs $1.5k$ modular multiplications, where k is the bit-length of the exponent E for the binary method. The modular multiplications for Walter's method [21] and Lou-Wu's method [28] are needed $\lceil \log_2 k \rceil * \frac{4}{3}$ and $0.689k + 11$ multiplications respectively. For comparison, all the methods mentioned in [1, 14, 21-27] are illustrated in Table 1. For a random exponents, the Walter's method takes 12 modular multiplications to evaluate M^e on average. Various representations of exponent e have been suggested with the same goal of reducing the number of multiplications involved. These presented algorithms [13-15] to evaluate the modular exponentiation computation $M^E \bmod N$ are showed in Table 2 for the practical application to public cryptography, where k is the bit-length of binary exponent E .

5. Conclusions and Future Works

We know many researchers are devoted to reducing the number of multiplication in computer algorithms for information management and network security usages. For example, Montgomery proposed an efficient algorithm for modular multiplications without trial division. This can be simplified as the operations of modular multiplication. Owing to the characteristic of no trial division in the Montgomery algorithm, the algorithm can be easily implemented in computer hardware or efficient software programs.

In this paper, we investigate many different methods to reduce the computational complexities for modern cryptosystems. Now there are still many novel methods [12, 30-32] issued in many computer security journals and reports for computer arithmetic operations and theoretical analyses. In the future, we can incorporate modular arithmetic and some novel methods (including hardware and software designs) to reduce the number of multiplications for modern cryptographic applications.

Table 1. Comparisons for computational complexities of modular multiplications.

Items	Methods	Modular multiplications
I	Walter's method	$\lceil \log_2 k \rceil * \frac{4}{3}$
II	Lou-Wu's method	$0.689k + 11$
III	Wu-Chang's method	$1.219k$
IV	Yen's method	$1.292k + 11$
V	Dimitrov and Jullien's method combining using joint sparse form	$1.500k$
VI	Yang-Guan-Laih's method	$1.509k$
VII	Dimitrov-Jullien's method	$1.534k$
VIII	Shamir's method combining minimal weight binary signed-digit recoding method	$1.556k$
IX	Pekmestzi's method	$1.75k$
X	Shamir's method	$1.75k$

Table 2. The average number of multiplications for different methods.

Items	multiplications	$k=512$	$k=1,024$	$k=2,048$	$k=4,096$	$k=8,192$	$k=16,834$
I	$\lceil \log_2 k \rceil * \frac{4}{3}$	12	13.333	14.667	16	17.333	18.667
II	$0.689k+11$	363.768	716.536	1,422.072	2,833.144	5,655.288	11,299.576
III	$1.219 k$	624.128	1,248.256	2,496.512	4,993.024	9,986.048	19,972.096
IV	$1.292 k + 11$	672.504	1,334.008	2,657.016	5,303.032	10,595.064	21,179.128
V	$1.5 k$	768	1,536	3,072	6,144	12,288	24,576
VI	$1.509 k$	772.608	1,545.216	3,090.432	6,180.864	12,361.728	24,723.456
VII	$1.534 k$	785.408	1,570.816	3,141.632	6,283.264	12,566.528	25,133.056
VIII	$1.556 k$	796.672	1,593.344	3,186.688	6,373.376	12,746.752	25,493.504
IX	$1.75 k$	896	1,792	3,584	7,168	14,336	28,672
X	$1.75 k$	896	1,792	3,584	7,168	14,336	28,672

References

- [1] Dimitrov V. S., Jullien G. A., and Miller W. C., "Complexity and fast algorithms for multi-exponentiations," *IEEE Transactions on Computers*, vol. 49, no. 2, 141-147, 2000.
- [2] Yang W.-C., Hsieh P.-Y., and Lai C.-S., "Efficient squaring of large integers," *IEICE Transactions on Fundamentals*, vol. E87-A., no. 5, 1189-1192, 2004.
- [3] Yen S.-M., Lai C.-S., and Lenstra A. K., "Multi-exponentiation," *IEE Proceedings Computer Digital Technology*, vol. 141, no. 6, 325-326, 1994.
- [4] Yamk T., Savas E., and Koc C. K., "Incomplete reduction in modular arithmetic," *IEE Proceedings: Computers and Digital Technique*, vol. 149, no. 2, 46-52, 2002.
- [5] Rivest R. L., Shamir A., and Adleman L., "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, 120-126, 1978.
- [6] Diffie W. and Hellman M. E., "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, 644-654, 1976.
- [7] Katti R. and Ruan X., "Left-to right binary signed-digit recording for elliptic curve cryptography," *Proceedings of the 2004 International Symposium on Circuit and Systems*, vol. 2, 365-368, 2004.
- [8] Nedjih N., "Efficient pre-processing for large window-based modular exponentiation using ant colony," *Informatica*, vol. 29, 155-161, 2005.
- [9] Khabbazzian M. and Gulliver T. A., "A new minimal average weight representation for left-to-right point multiplication methods," *IEEE Transactions on Computers*, vol. 54, no. 11, 1454-1459, 2005.
- [10] Knuth D. E. *The Art of Computer Programming*, vol. II: *Seminumerical Algorithms*. 3rd Edition, Addison-Wesley, MA., 1997.
- [11] Guan D. J., "Montgomery algorithm for modular multiplication," *IEEE Transactions on Computers*, vol. 42, no. 6, 693-699, 2003.
- [12] Hassan M. A., "Efficient computation of multiplicative inverses for cryptographic applications," *Proceedings of the 15th IEEE Symposium on Computer Arithmetic*, 66-72, 2001.
- [13] Lai Y.-P. and Chang C.-C., "An efficient multi-exponentiation scheme based on modified Booth's method," *Journal of Electronics Engineering*, vol. 90, no. 3, 221-233, 2003.
- [14] Yang W.-C., Guan D. J., and Lai C.-S.,

- “Algorithm of asynchronous binary signed-digit recording on fast multi-exponentiation,” *Applied Mathematics and Computation*, vol. 167, 108-117, 2005.
- [15] Lou D.-C. and Wu C.-L., “Parallel exponentiation using common- multiplicand-multiplication and signed- digit-folding techniques,” *International Journal of Computer Mathematics*, vol. 81, no. 10, 1187-1202, 2004.
- [16] Ha J.-C. and Moon S.-J., “A common-multiplicand method to the Montgomery algorithm for speeding up exponentiation,” *Information Proceeding Letters*, vol. 66, 105-107, 1998.
- [17] Joye M. and Yen S.-M., “Optimal left-to-right binary signed-digit recording,” *IEEE Transactions on Computers*, vol. 49, no. 7, 740-748, 2000.
- [18] Martinez M. A. G., Luna G. M., and Henriquez F. R., “Hardware implementation of the binary method for exponentiation in $GF(2^m)$,” *Proceedings of the 4th Mexican International Conference on Computer Science*, 131-134, 2003.
- [19] Koc C. K. and Johnson S., “Multiplication of signed-digit numbers,” *Electronics Letters*, vol. 30, no. 11, 840-841, 1994.
- [20] Montgomery P. L., “Modular multiplication without trial division,” *Mathematics of Computation*, vol. 44, no. 170, 519-521, 1985.
- [21] Eldridge S. E. and Walter C. D., “Hardware implementation of Montgomery’s modular multiplication algorithm,” *IEEE Transactions on Computers*, vol. 42, no. 6, 693-699, 1993.
- [22] Yen S.-M. and Lai C.-S., “Common-multiplicand multiplication and its applications to public key cryptography,” *Electronics Letters*, vol. 29, no. 17, 1583-1584, 1993.
- [23] Koc C. K., “Analysis of sliding window techniques for exponentiation,” *Computer Mathematics Application*, vol. 30, no. 10, 17-24, 1995.
- [24] Lou D.-C. and Chang C.-C., “Fast exponentiation method obtained by folding the exponent in Half,” *Electronics Letters*, vol. 32, no. 11, 984-985, 1996.
- [25] Yen S.-M., “Improved common- multiplicand multiplication and fast exponentiation by exponent decomposition,” *IEICE Transactions on Fundamentals*, vol. E80-A, no. 6, 1160-1163, 1997.
- [26] Wu T.-C. and Chang Y.-S., “Improved generalization common-multiplicand multiplications algorithm of Yen and Lai,” *Electronics Letters*, vol. 31, no. 20, 1738-1739, 1995.
- [27] Arno S. and Wheeler F. S., “Signed digit representations of minimal hamming weight,” *IEEE Transactions on Computers*, vol. 42, no. 8, 1007-1010, 1993.
- [28] Lou D.-C., Wu C.-L., and Chen C.-Y., “Fast exponentiation by folding the signed-digit exponent in half,” *International Journal of Computer Mathematics*, vol. 80, no. 10, 1251-1259, 2003.
- [29] Koc C. K., “High-radix and bit recording techniques for modular exponentiation,” *International Journal of Computer Mathematics*, vol. 40, 139-156, 1991.
- [30] Garbner P. J., Heuberger C., and Prodinger H., “Distribution results for low-weight binary representations for pairs of integers,” *Theoretical Computer Science*, vol. 319, 307-331, 2004.
- [31] Negre, C., “Quadrinomial modular arithmetic using modified polynomial basis,” *Proceedings of 2005 International Conference on Information Technology: Coding and Computing*, vol. 1, 550-555, 2005.

- [32] Dempster A. G. and Macleod M. D., “Generation of signed-digit representations for integer multiplication,” *IEEE on Signal Processing Letters*, vol. 11, no. 8, 663-665, 2004.